Emoji Polarity Classification in Twitter Text

Elyas El Khaldi, Majd El Blidi, Mohamed Abbes
Department of Computer Science, EPFL, Switzerland

***Abstract*—NLP classification analysis requires processing millions of tweets, but state-of-the-art transformer models impose prohibitive computational costs.We address this gap by systematically investigating whether carefully designed lightweight models can approach transformer performance at dramatically lower cost. Through controlled experiments on 2.5M tweets, we demonstrate that a BiLSTM with Twitter-specific preprocessing achieves near 86% accuracy , reaching 95% of BERTweet's performance with 30× fewer parameters and 3× faster training. These findings demonstrate that resource-efficient models remain viable alternatives to transformers for emoticon polarity classification when domain-specific optimizations are carefully applied.**

I. Introduction

The aim of this project is to classify tweets from a large-scale Twitter dataset in which two sentiment-bearing emoticons, :) and :(, have been removed from the text while every tweet was labeled based on the previous emoticon (1 for :) and -1 for :( ). This setting is challenging due to the noisy and informal nature of Twitter language (slang, creative spelling, emotive punctuation, and repeated characters), which makes robust text classification non-trivial. Importantly, the supervision signal captures emoticon polarity rather than the full semantic sentiment of the tweet, which motivates careful processing of text data.

We study how modeling choices affect performance on this task by systematically comparing simple linear baselines with more expressive neural approaches. In particular, we investigate the role of Twitter-specific preprocessing and vocabulary pruning, different embedding initialization strategies for word-based models, and the impact of model architecture—from convolutional and recurrent networks to a pre-trained transformer (BERTweet [2]).

The remainder of this paper is organized as follows: Section II details the dataset and preprocessing pipeline. Section III presents our embedding strategy. Section IV describes the architectures used. Section V outlines the training procedure. Section VI presents experimental results and ablation studies. Section VII discusses findings, and Section VIII concludes the study.

II. Dataset and Preprocessing

A. Dataset Description

The training dataset contains 2,500,000 tweets, perfectly balanced with 1,250,000 positive and negative samples each. The data has been pre-processed with user mentions anonymized to <user> tags and URLs replaced with <url> tags. All characters are lowercase, and only the target emoticons ':)' and ':(' have been removed, while other emoticons (e.g., ';)') remain in the text.

Before preprocessing, we conducted a lightweight exploratory analysis to better understand the data characteristics. We observed a heavy-tailed distribution of tweet lengths, with most tweets containing fewer than 25 tokens. Token frequency analysis revealed a large long-tail vocabulary driven by slang, creative spellings, and emojis, justifying aggressive normalization and vocabulary reduction. Class distributions were perfectly balanced by construction, removing the need for class reweighting.

B. Preprocessing Pipeline

Raw tweets contain significant noise that requires careful processing. Our preprocessing pipeline includes:

- **Duplicate Removal:** Strictly identical tweets are removed from training and validation sets to prevent frequency bias.
- **Contraction Expansion:** Common english language contractions ("can't" → "cannot", "won't" → "will not", etc.) are expanded to their full forms to improve vocabulary consistency.
- **Emoticon Handling:** Emoticons are explicitly mapped to sentiment tokens: positive emoticons (^^, :D, =)) become `<SMILE>`, while negative emoticons (:{, :¡) become `<SAD>`.
- **Punctuation Handling:** All punctuation symbols are removed during preprocessing, except for ellipses ("..."), exclamation marks ("!"), and question marks ("?"). Ellipses are replaced by the special token `<DOTS>`, while "!" and "?" are preserved as they often carry strong sentiment information in tweets.
- **Character Repetition:** Words with repeated characters (e.g., "coool") are normalized to their base form and tagged with repetition count ("cool `<REP_3>`") to preserve intensity information.
- **Word Repetition:** Consecutively repeated words are replaced with a single instance plus a repetition tag (`<REPEAT_N>`).
- **Negation Tagging:** Words appearing after negation terms ("not", "never", etc.) are suffixed with `_NEG` until the next punctuation mark, helping models capture negation of words.
- **Number Normalization:** All numerical values are replaced by the special token `<NUM>` to reduce vocabulary size.

After preprocessing, vocabulary size was reduced from 418,634 unique tokens to 117,749 tokens (using a minimum frequency threshold of 3), while preserving task-relevant features.

C. Preprocessing for BERTweet

Given that the pre-trained model BERTweet [2] was trained on a highly noisy Twitter dataset, we tried to keep that noise in the data that we fed the fine-tuner. We aligned the existing dataset normalization with the preprocessing scheme used during BERTweet pre-training.

1) **User mention:** User mentions in our dataset are replaced by the <user> tag that we translated into a @USER tag like in the BERTweet normalization
2) **URLS:** URLs in our dataset are replaced by the <url> tag that we translated into 'HTTPURL'
3) **TweetNormalizer:** we use normalizer that is provided in the BERTweet library to clean the rest of the text.

For BERTweet, we rely on the model's recommended Twitter-specific normalization pipeline. As a result, performance differences between BERTweet and the other neural architectures combine both architectural effects and preprocessing choices.

III. Embedding Strategies

Word embeddings played a central role in our text classification task. It maps discrete tokens to continuous representations of which features can be utilized. In this project, we compare the performance of pre-trained embeddings with embeddings trained solely on the training corpus, in order to compute how a larger embedding space can be utilized to improve performance.

A. Trained Embeddings

We first try to train GloVe [1] embeddings on the entire tweet dataset by constructing co-occurence of tokens matrix.

1) **Co-occurrence Matrix Construction:** We compute a word-word co-occurrence matrix X from our training corpus using an optimized sparse matrix approach:
   - Used CountVectorizer to create document-term matrix
   - Computed co-occurrence as $X^T \cdot X$
   - Resulted in 41,380,020 non-zero entries
2) **Embedding Training:** We train embeddings on the vocabulary found in the dataset by minimizing the GloVe loss function:

$$J = \sum_{i,j=1}^{V} f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2 \quad (1)$$

   where $f(X_{ij}) = \min(1, (X_{ij}/n_{\max})^\alpha)$ is a weighting function with $n_{\max} = 100$ and $\alpha = 0.75$.

We train embeddings of size 50 by running the algorithms for 10 epochs.

B. Pretrained GloVe embeddings

In addition to corpus-trained embeddings, we evaluate pre-trained GloVe embeddings trained on large-scale Twitter data. Specifically, we use the Twitter version of GloVe embeddings introduced by Pennington [5], which are learned from a much larger and linguistically diverse Twitter corpus. This comparison allows us to assess whether external linguistic information captured from large-scale data provides a measurable benefit over embeddings learned exclusively from the task-specific dataset.

IV. Models

To assess the impact of model architecture and representation learning choices, we implemented and compared several baseline and advanced models, with particular focus on BiLSTM as a resource-efficient alternative to transformers. We include BERTweet as an upper-bound reference to quantify the performance-cost trade-off. All models use the same train-validation splits (90

A. Baseline Model

1) Logistic Regression: A linear classifier trained on tweet-level representations obtained by averaging corpus trained GloVe embeddings. This approach ignores word order but provides a baseline that captures global semantic information.

B. Advanced Models

1) Convolutional Neural Network (CNN): We implemented a deep sequential CNN designed to capture hierarchical patterns. Unlike the parallel baseline, our model stacks layers to refine feature extraction:

- **Feature Extraction:** Two stacked Conv1D layers. The first uses 128 filters ($k = 3$) with Batch Normalization to capture broad semantic phrases. The second uses 128 filters ($k = 5$) for refined local patterns.
- **Regularization:** To handle Twitter noise, we applied Dropout with rate $p = 0.3$ in the final 64-unit dense layer.
- **Global Pooling:** Global MaxPooling1D extracts the most salient features before the final Sigmoid classification.

For all convolutional layers, we used stride 1 with same-padding to preserve sequence length across layers.

2) Bidirectional LSTM (BiLSTM): We implemented a multi-layer Bidirectional LSTM to capture long-range dependencies and context from both directions of the tweet text.

- **Embedding Layer:** Initialized with 50-dimensional pre-trained embeddings and set to non-frozen (*freeze=False*) to allow continuous domain adaptation during training.
- **Recurrent Encoder:** A 2-layer BiLSTM with a hidden dimension of 128. This configuration captures deeper semantic relations than a single-layer model while maintaining computational efficiency.
- **Feature Aggregation:** We concatenate the final hidden states from both the forward and backward passes of the last layer, resulting in a 256-dimensional feature vector.

- **Classification Head:** Features are processed through a Dropout layer ($p = 0.5$), a 64-unit ReLU dense layer, and a final Sigmoid neuron for binary classification prediction.

3) BERTweet [2]: We fine-tuned the `vinai/bertweet-base` model, a RoBERTa-based [3] transformer pre-trained on 850 million English tweets using Byte-Pair Encoding (BPE) tokenization and Twitter-specific normalization. We include it primarily as a strong reference point rather than a lightweight deployable solution. Fine-tuning details:

- **Model:** BERTweet-base (125M parameters)
- **Max sequence length:** 128 tokens
- **Batch size:** 128 with gradient accumulation
- **Learning rate:** 3×10^{-5} with linear warmup (warmup ratio : 0.06)
- **Epochs:** 3
- **Optimization:** AdamW with weight decay 0.01

The final classification token representation is used for binary prediction.

V. Training Procedure

A. Data Split

The 2.5M tweet dataset was split into 90% training (2,250,000 samples) and 10% validation (250,000 samples). Random shuffling ensured balanced class distribution.

B. Validation Analysis

Hyperparameters were selected using a held-out validation set for BiLSTM due to computing while k-fold cross-validation was performed rather for CNN. The dataset was split into 90% training and 10%validation data, preserving class balance through stratified sampling.

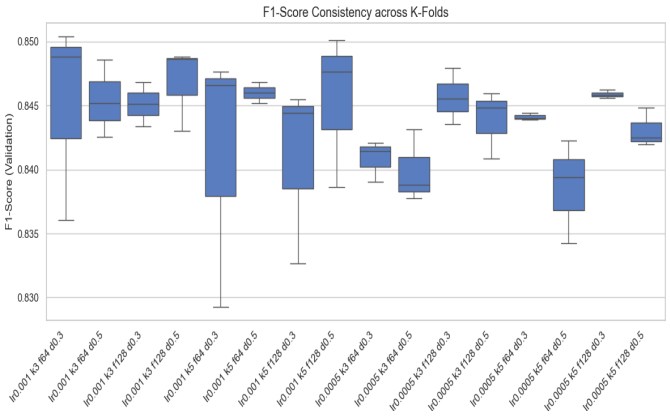

Fig. 1: Validation F1-score distribution across K-folds for different CNN hyperparameter configurations. (lr, kernel size, filters, dropout)

Due to the computational cost of fine-tuning large transformer models, only certain fixed parameters were tested for BERTweet.

C. Optimization Settings

Neural models were trained with:

- **Optimizer:** Adam
- **Loss Function:** CrossEntropyLoss and BCE for Bi-LSTM
- **Early Stopping:** Based on validation accuracy
- **Device:** NVIDIA L4 GPU with CUDA
- **Random seed:** 42, fixed across all experiments for reproducibility

Model	Batch Size	LR	Epochs
Logistic Reg.	-	0.001	-
CNN	64	0.001	3
BiLSTM	64	0.001	3
BERTweet	128	3×10^{-5}	3

TABLE I: Hyperparameters for each model architecture.

D. Embedding Fine-Tuning Strategy

For CNN and BiLSTM models, embeddings are unfrozen so they can be fine-tuned throughout the training

E. Regularization

- Dropout applied to embedding and hidden layers (p=0.3 for CNN and p=0.5 for BiLSTM)
- Gradient clipping (max norm = 1.0) for BERTweet
- Weight decay (0.01) for BERTweet

F. Training Dynamics

To analyze optimization stability and generalization behavior, we monitored training and validation performance across epochs for our main BiLSTM model.

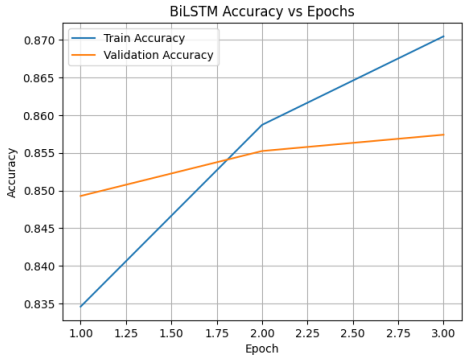

Fig. 2: The curves are shown for a single representative train–validation split using the best hyperparameter configuration selected via grid-search validation. The small gap between curves indicates limited overfitting and stable convergence.

As shown in Figure 2, the model converges within a small number of epochs, and validation performance closely follows training performance.

VI. Results

A. Model Comparison

We use logistic regression with averaged GloVe embeddings as a strong yet simple baseline, representing a non-sequential model that ignores word order. All subsequent architectures are evaluated relative to this baseline to quantify the benefit of increased model capacity and representation learning.

Model	Val. Acc.	F1-score
Logistic Regression	59.1%	0.596
CNN	85.24%	0.85
BiLSTM	85.8%	0.8613
BERTweet	92.1%	0.92

TABLE II: Performance comparison across all models on the validation set.

Key observations:

- The relatively low performance of logistic regression reflects its reliance on averaged word embeddings, which discards word order and interaction effects that are crucial in informal Twitter text.
- BERTweet achieves highest single-model performance but at 50× computational cost

B. Ablation Studies

Configuration	Val. Acc.	F1-score
No preprocessing	73.1%	0.755
With preprocessing	**84%**	**0.846**

TABLE III: Impact of preprocessing on BiLSTM. Test set

1) Effect of Preprocessing:

Embedding Type	Val. Acc.	F1-score
GloVe (corpus trained)	84.28%	0.8690
GloVe (pretrained)	85.8%	0.8613

TABLE IV: Effect of embedding initialization on BiLSTM

2) Effect of Embedding Strategy:

C. Training Efficiency

Model	Params	Train Time	Accuracy
CNN	8.4M	25 min	85.24%
BiLSTM	4.2M	30 min	85.8%
BERTweet	125M	95 min	92%

TABLE V: Model complexity and training time (per epoch).

VII. Discussion

A. Key Findings

Our experiments demonstrate that model performance depends on three factors: (1) preprocessing quality, (2) embedding initialization, and (3) architectural capacity. Each ablation component contributes meaningfully, with preprocessing and embeddings accounting for the largest gains.

B. Model Selection Trade-offs

While transformer-based models (BERTweet) achieve superior results, they require:

- 30× more parameters (125M vs. 4.2M)
- 3× longer training time
- Specialized hardware for efficient training

We include BERTweet primarily as a reference upper bound to contextualize the performance of lighter architectures rather than as a deployable solution.

For resource-constrained scenarios, BiLSTM with pretrained embeddings provides an excellent balance of performance and efficiency.

C. Limitations

- While implemented sparsely, the co-occurency matrix construction still scales poorly with vocabulary size.
- Domain-specific embeddings may not generalize to other text sources

D. Error Analysis

Qualitative inspection of misclassified tweets reveals that most errors arise from sarcastic expressions, pragmatic emoji usage, and long-range negation scopes that exceed the model's effective context window. In particular, tweets where emojis contradict textual sentiment (e.g., ironic positive emojis in negative statements) remain challenging for all non-transformer models. These observations suggest that richer contextual modeling and pragmatic understanding are key remaining challenges in emoji polarity classification.

VIII. Conclusion

We conducted a systematic experimental study of emoticon classification on Twitter data, comparing multiple preprocessing strategies, embedding choices, and model architectures. Our experiments demonstrate that careful preprocessing and embedding adaptation substantially improve the performance of lightweight neural models. While transformer-based models achieve the best absolute results, recurrent models offer a strong trade-off between accuracy and computational efficiency. Overall, this project highlights the importance of controlled experimentation and ablation analysis when applying machine learning models to noisy real-world text data. Our results show that careful preprocessing, corpus-adapted embeddings, and neural network models can achieve strong performance (around 86% accuracy) while remaining significantly more efficient than transformer-based approaches.

IX. Ethical study

A. Risk of Misinterpreting Emoticon Polarity as Sentiment.

A potential ethical risk in this project is the misinterpretation or reuse of our classifier as a general sentiment (or emotion) detector. Our task is more specific: the dataset provides binary labels derived solely from the presence of two sentiment-bearing emoticons (:) and :(), which are removed from the tweet text before training. Consequently, the learned signal targets emoticon polarity rather than the overall sentiment expressed by the tweet. The main affected stakeholders are downstream users (researchers, developers, practitioners) who may deploy the model in decision-making pipelines (e.g., moderation, opinion mining, user profiling) without understanding this supervision signal; indirectly, social media users may be impacted by incorrect inferences about their opinions or mood. Severity can be moderate to high in aggregate (systematic mislabeling at scale), and likelihood is non-negligible because high-performing text classifiers are often assumed to perform sentiment analysis by default.

B. Empirical Evaluation of the Risk and Mitigation Measures.

We evaluated this risk quantitatively by comparing our polarity labels to predictions from a strong, independently trained Twitter sentiment oracle (three-way: negative/neutral/positive) [4]. When forcing the oracle into a binary decision by mapping neutral to a polarity (via pos-vs-neg logits), agreement with emoticon polarity is only Acc = 0.6837 / F1 = 0.7084 on the 200k split, with very similar results on the full 2.5M dataset (Acc = 0.6824 / F1 = 0.7071), indicating a stable mismatch between emoticon-derived polarity and sentiment predictions at scale. Crucially, when keeping the oracle's neutral class explicit, 43.8% of the 200k tweets are predicted to be neutral (87,592 / 200,000). In these cases, there is no clear sentiment signal in the text according to the oracle, yet the dataset enforces a binary label—meaning that if our model were (mis)used as a sentiment classifier, its output on many neutral tweets would effectively resemble a coin flip rather than a meaningful sentiment judgment. We account for this risk by (i) clearly scoping the project as emoticon polarity classification, (ii) reporting the above oracle-based evidence in the README.

References

[1] J. Pennington, R. Socher, and C. Manning, "GloVe: Global Vectors for Word Representation," in *Proceedings of EMNLP*, 2014, pp. 1532–1543.

[2] D. Q. Nguyen, T. Vu, and A. T. Nguyen, "BERTweet: A pre-trained language model for English Tweets," in *Proceedings of EMNLP*, 2020.

[3] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv preprint arXiv:1907.11692*, 2019.

[4] Cardiff NLP, "Twitter RoBERTa-base for Sentiment Analysis," HuggingFace model card, 2020.

[5] J. Pennington, R. Socher, and C. Manning, "GloVe: Global Vectors for Word Representation (Twitter version)," Stanford NLP, 2014.